Compiler Design Project — Complete Code Explanation

> **Language:** C++ | **Target:** LLVM IR | **Custom Language Features:** variables, functions, blueprints (classes), loops, print/scan

---

Table of Contents

---

1. Project Overview & Pipeline

Your compiler takes source code written in your custom language and transforms it through **5 major stages**:

```
Source Code (text)
      ↓
   [LEXER]        → Breaks text into Tokens
      ↓
   [PARSER]       → Builds an Abstract Syntax Tree (AST)
      ↓
 [AST PRINTER]    → Prints the tree (for debugging)
      ↓
[SEMANTIC ANALYZER] → Checks types, scopes, logic errors
      ↓
 [IR GENERATOR]   → Converts AST to LLVM Intermediate Representation (IR)
      ↓
   LLVM IR (machine-readable code that can be compiled to binary)
```

Each file plays one role in this pipeline. Let's go through every file in order.

---

2. Token.h — The Token System

This is the **foundation** of the entire compiler. Everything else depends on it.

What is a Token?

A token is the smallest meaningful unit of your source code. For example, the code `let x: int = 5;` produces tokens: `LET`, `VAL("x")`, `COLON`, `INT`, `ASSIGN`, `INT_LIT("5")`, `SEMICOLON`.

Line-by-Line: `enum class TokenType`

```cpp
enum class TokenType {
```

Defines all possible token categories. `enum class` means it's **strongly typed** — you can't accidentally mix up token types.

```cpp
    PLUS, MINUS, DIV, MULTI, MOD,
```

Basic arithmetic operators: `+`, `-`, `/`, `*`, `%`

```cpp
    VAL, ERROR, OP_QUESTION, OP_COLON,
```

- `VAL` = a user-defined identifier (variable/function name)
- `ERROR` = something unrecognized
- `OP_QUESTION` = `?` (for ternary operator)
- `OP_COLON` = `:` (used in type annotations like `let x: int`)

```cpp
    IF, ELSE, WHILE, FOR, INT, FLOAT, DOUBLE, STRING, CHAR, BOOLEAN,
    RETURN, CONTINUE, BREAK, NILL, VOID, LET, FUNCTION, NEW,
```

All **keywords** of your custom language. Each keyword gets its own token type so the parser can identify them.

```cpp
    INT_LIT, FLOAT_LIT, DOUBLE_LIT, STRING_LIT, CHAR_LIT, BOOLEAN_LIT,
```

Literal value tokens — the actual data values written in source code like `42`, `3.14`, `"hello"`, `'c'`, `true`.

```cpp
    EQUALS, ASSIGN, PLUS_EQU, SUB_EQU, DIV_EQU, MULTI_EQU, MOD_EQU,
    AND, OR, NOT, POINTER, ARRAY,
    GREATER, LESSER, GREATER_EQU, LESSER_EQU, NOT_EQU,
    POST_INCREMENT, POST_DECREMENT, ARROW, PRE_INCREMENT, PRE_DECREMENT,
```

- `ASSIGN` = `=` (assignment), `EQUALS` = `==` (comparison)
- `PLUS_EQU` = `+=`, `SUB_EQU` = `-=`, etc. (compound assignments)
- `AND` = `&&`, `OR` = `||`, `NOT` = `!`
- `POINTER` and `ARRAY` = used in type declarations like `int*` or `int[5]`
- `ARROW` = `->` (for pointer member access)

```cpp
    LS_ASSIGN, RS_ASSIGN, BIT_AND_ASSIGN, BIT_XOR_ASSIGN, BIT_OR_ASSIGN,
    BIT_OR, BIT_XOR, BIT_AND, L_SHIFT, R_SHIFT,
```

Bitwise operators: `|`, `^`, `&`, `<<`, `>>` and their compound assignment forms.

```cpp
    CLASS, MEMBER, PUBLIC, PROTECTED, PRIVATE,
```

Object-oriented tokens. `CLASS` maps to your `blueprint` keyword.

```cpp
    L_PAREN, L_SQUB, L_CURLB, R_PAREN, R_SQUB, R_CURLB,
```

Bracket tokens: `(`, `[`, `{`, `)`, `]`, `}`

```cpp
```

```
    COMMENT, SEMICOLON, DOT, COMMA,
    TYPECAST, Block, EXPR_STMT,
    EF,
    PRINT, SCAN
```

- `COMMENT` = `//`
- `EF` = End of File (signals the end of input)
- `PRINT` and `SCAN` = your custom I/O keywords

Line-by-Line: `struct Token`

cpp

```cpp
struct Token {
    TokenType type;
    std::string_view value;
};
```

A token has exactly two fields:

- `type` — which category it belongs to (from the enum above)
- `value` — the actual text from the source code (e.g., `"hello"` or `"42"`)

`string_view` is used (not `string`) because it's a **lightweight view into the source string** — no copying is needed, which is efficient.

Line-by-Line: `struct Type`

This represents a **data type** in your language (like `int`, `int*`, `int[5]`, or a custom Blueprint).

cpp

```cpp
struct Type {
    TokenType base;
    Type* innerType = nullptr;
    std::string structName = "";
    int arraySize = -1;
```

- `base` — the base kind: INT, FLOAT, POINTER, ARRAY, CLASS, etc.

- `innerType` — for nested types. A pointer to int has `base=POINTER, innerType→{base=INT}`. This allows types like `int**` (pointer to pointer).
- `structName` — for Blueprints (classes), stores the name like `"Player"`
- `arraySize` — for arrays like `int[10]`, stores `10`. `-1` means unknown/dynamic size.

```cpp
    Type() : base(TokenType::ERROR) {}
    Type(TokenType b) : base(b) {}
    Type(TokenType b, Type* inner, int size=-1) : base(b), innerType(inner), arraySi
    Type(TokenType b, std::string name) : base(b), structName(name) {}
```

Four constructors for convenience:

1. Default → ERROR type
2. Simple type → just a base (e.g., `Type(TokenType::INT)`)
3. Pointer/Array type → base + inner type + optional size
4. Blueprint type → base + struct name

```cpp
    bool operator==(const Type& other) const {
        if (base != other.base) return false;
        if (structName != other.structName) return false;
        if (arraySize != other.arraySize) return false;
        if (innerType && other.innerType) return *innerType == *other.innerType;
        return innerType == other.innerType;
    }
    bool operator!=(const Type& other) const { return !(*this == other); }
```

Custom equality check — compares all fields **recursively**. This is crucial for the SemanticAnalyzer to catch type mismatches. A `int*` must not equal `int` or `float*`.

---

3. Lexer.h & Lexer.cpp — The Lexer

The Lexer reads raw source code text and breaks it into a stream of Tokens. It's the **first stage** of the compiler.

Lexer.h — The Header

```cpp
class Lexer {
private:
    std::string_view src;    // The entire source code (no copying)
    size_t pos = 0;          // Current position in the source
    std::unordered_map<std::string_view, TokenType> keywords;  // keyword lookup tab
```

- `src` — holds a view of the full source code text
- `pos` — a cursor that moves through the source, character by character
- `keywords` — a hash map for O(1) keyword lookup. When we see `if`, we look it up here instead of comparing to every possible keyword.

```cpp
    char peek();      // Look at current char without moving
    char peekNext();  // Look at next char without moving
    char advance();   // Read current char and move forward
```

Three helper functions that simulate a **reading cursor**. This pattern is standard in lexers.

```cpp
public:
    Lexer(std::string_view v);
    Token getNextToken();   // The main function — returns the next token
```

Lexer.cpp — The Implementation

Constructor

```cpp
```

```cpp
Lexer::Lexer(std::string_view v) : src(v) {
    keywords = {
        {"if",        TokenType::IF},
        {"else",      TokenType::ELSE},
        {"blueprint", TokenType::CLASS},   // 'blueprint' is your custom class keywo
        {"function",  TokenType::FUNCTION},
        {"let",       TokenType::LET},
        {"print",     TokenType::PRINT},
        {"scan",      TokenType::SCAN},
        {"true",      TokenType::BOOLEAN_LIT},
        {"false",     TokenType::BOOLEAN_LIT},
        // ... and all other keywords
    };
}
```

Initializes `src` and fills the keyword map. Note that `blueprint` maps to `CLASS` — your language uses `blueprint` instead of `class`.

Helper Functions

cpp

```cpp
char Lexer::peek()     { return (pos < src.size()) ? src[pos] : '\0'; }
char Lexer::peekNext() { return (pos+1 < src.size()) ? src[pos+1] : '\0'; }
char Lexer::advance()  { return src[pos++]; }
```

- `peek()` — returns current char without consuming it. Returns `\0` (null) at end of file.
- `peekNext()` — looks one character ahead without consuming. Essential for two-character tokens like `==`, `+=`, `->`.
- `advance()` — returns and consumes the current character (moves `pos` forward).

`getNextToken()` — The Main Lexer Function

cpp

```cpp
Token Lexer::getNextToken() {
    while (pos < src.size() && isspace(src[pos])) pos++;
```

Skip whitespace — spaces, tabs, and newlines are not tokens; skip all of them first.

cpp

```cpp
    if (pos >= src.size()) return {TokenType::EF, ""};
```

If we've consumed all input, return **End of File** token.

```cpp
    size_t start = pos;
    int state = 0;
```

`start` remembers where the current token began (for extracting `string_view` later). `state` is the **state machine variable** — this lexer is implemented as a **Finite State Machine (FSM)**.

The State Machine — `switch(state)`

State 0: Dispatch (decide what kind of token this is)

```cpp
case 0:
    if (isalpha(c)) { state = 1; advance(); }       // → identifier/keyword
    else if (c == '"') { state = 3; advance(); }     // → string literal
    else if (c == '\'') { state = 4; advance(); }    // → char literal
    else if (isdigit(c)) { state = 2; advance(); }   // → number literal
```

The first character determines which state to enter.

Two-character operators (like `==`, `!=`, `+=`):

```cpp
    else if (c == '=') {
        advance();
        if (peek() == '=') { advance(); return {TokenType::EQUALS, "=="}; }
        return {TokenType::ASSIGN, "="};
    }
```

After consuming `=`, peeks at the next character. If it's also `=`, returns `==`. Otherwise returns `=`. This is the pattern used for all multi-character operators (`->`, `>=`, `<=`, `+=`, `-=`, `//`, etc.).

State 1: Identifiers and Keywords

```cpp
```

```
case 1: {
    while (isalnum(peek())) advance();  // Consume all letters and digits
    std::string_view text = src.substr(start, pos - start);  // Extract the word
    auto it = keywords.find(text);
    if (it != keywords.end()) return {it->second, text};  // It's a keyword!
    return {TokenType::VAL, text};  // It's a variable/function name
}
```

Reads until a non-alphanumeric character is found, then checks if it's a keyword. If yes, returns the keyword token; otherwise it's a user identifier (`VAL`).

State 2: Number Literals

cpp

```
case 2: {
    bool isFP = false;
    while (isdigit(peek())) advance();  // Consume integer digits
    if (peek() == '.' && isdigit(peekNext())) {
        isFP = true;
        advance();
        while (isdigit(peek())) advance();  // Consume decimal digits
    }
    if (peek() == 'f' || peek() == 'F') {
        advance();
        return {TokenType::FLOAT_LIT, src.substr(start, pos - start)};
    }
    return {isFP ? TokenType::DOUBLE_LIT : TokenType::INT_LIT, src.substr(start, pos
}
```

Handles integers (`42`), doubles (`3.14`), and floats (`3.14f`). The `isFP` flag tracks whether a decimal point was seen.

State 3: String Literals

cpp

```cpp
case 3: {
    while (peek() != '"' && peek() != '\0') advance();  // Consume until closing quo
    if (peek() == '\0') return {TokenType::ERROR, "Unterminated string"};
    advance();  // Consume closing "
    return {TokenType::STRING_LIT, src.substr(start, pos - start)};
}
```

Reads everything between the opening and closing `"`. Returns an error if the string is never closed.

State 4: Char Literals

```cpp
case 4: {
    if (peek() != '\'' && peek() != '\0') advance();  // Consume the one character
    if (peek() == '\'') { advance(); return {TokenType::CHAR_LIT, src.substr(start,
    return {TokenType::ERROR, "Unterminated char"};
}
```

A char literal is `'x'` — reads exactly one character between single quotes.

---

4. parser.h & parser.cpp — The Parser

The Parser takes the flat stream of tokens from the Lexer and organizes them into a **tree structure** (the AST — Abstract Syntax Tree). This tree reflects the logical structure of your program.

---

parser.h — The Header

```cpp
class Parser {
private:
    vector<Token> tokens;   // All tokens stored at once
    size_t pos = 0;         // Current position in the token list
```

Unlike the Lexer which processes one character at a time, the Parser works on a pre-filled `vector<Token>`. The entire token stream is passed in at once.

```cpp
    Token peek();       // Look at current token
    Token consume();    // Read and advance
    bool is_at_end();   // Check if all tokens consumed
    void throw_error(string_view msg);
    void consume_if(TokenType type, string_view error_msg);  // Expect a specific to
```

```cpp
    Type parse_type();  // Parse a type annotation like "int", "float*", "int[5]"

    unique_ptr<Stmt> parse_statement();
    unique_ptr<Stmt> parse_block();
    unique_ptr<Stmt> parse_if();
    unique_ptr<Stmt> parse_while();
    unique_ptr<Stmt> parse_for();
    unique_ptr<Stmt> parse_return();
    unique_ptr<Stmt> parse_break();
    unique_ptr<Stmt> parse_continue();
    unique_ptr<Stmt> parse_variable_declaration();
    unique_ptr<Stmt> parse_function();
    unique_ptr<Stmt> parse_blueprint();
    unique_ptr<Stmt> parse_print();
    unique_ptr<Stmt> parse_scan();
    unique_ptr<Expr> parse_expression(int min_bp = 0);
```

One `parse_*` function per language construct. They all return `unique_ptr` (smart pointers) so memory is managed automatically — no manual `delete` needed.

---

parser.cpp — The Implementation

Operator Precedence Table

```cpp
static const std::unordered_set<TokenType> assign = {
    TokenType::ASSIGN, TokenType::PLUS_EQU, TokenType::SUB_EQU, ...
};
```

A set of all assignment operators for quick lookup.

This is the **Pratt Parser** operator precedence table. Each operator has a **left binding power** and a **right binding power**. Higher numbers = higher precedence. For example:

- `*` (25/26) binds tighter than `+` (23/24), so `2 + 3 * 4` becomes `2 + (3 * 4)`.
- Assignment `=` (4/3) is **right-associative** — the right power (3) is lower than left (4), making `a = b = c` parse as `a = (b = c)`.

Helper Functions

cpp

```cpp
Token Parser::peek()    { return (pos < tokens.size()) ? tokens[pos] : Token{TokenTy
Token Parser::consume() { if (pos >= tokens.size()) throw_error("Unexpected end"); r
bool Parser::is_at_end() { return peek().type == TokenType::EF; }
```

Same cursor pattern as the Lexer but operating on tokens instead of characters.

cpp

```cpp
void Parser::consume_if(TokenType type, string_view error_msg) {
    if (peek().type == type) consume();
    else throw_error(error_msg);
}
```

Expects a specific token. If found, consumes it. If not, throws a clear error. Used everywhere for syntax enforcement (e.g., expecting `;` after a statement).

`parse_type()` — Recursive Type Parser

cpp

```
Type Parser::parse_type() {
    Token t = consume();
    Type currentType;

    if (t.type == TokenType::INT || ...) currentType = Type(t.type);  // Base type
    else if (t.type == TokenType::VAL) currentType = Type(TokenType::CLASS, string(
    else throw_error("Expected a valid type");

    while (!is_at_end()) {
        if (peek().type == TokenType::MULTI) {
            consume();
            currentType = Type(TokenType::POINTER, new Type(currentType));  // Wrap
        } else if (peek().type == TokenType::L_SQUB) {
            consume();
            int size = -1;
            if (peek().type == TokenType::INT_LIT) size = stoi(string(consume().valu
            consume_if(TokenType::R_SQUB, "Expected ']'");
            currentType = Type(TokenType::ARRAY, new Type(currentType), size);  // W
        } else break;
    }
    return currentType;
}
```

Parses types **recursively** from the inside out. After reading the base type, it loops to check for `*` (pointer) or `[N]` (array). Each wraps the previous type as its inner type, enabling complex types like `int**` or `int[10]*`.

`parse_statement()` — The Statement Dispatcher

cpp

```
unique_ptr<Stmt> Parser::parse_statement() {
    Token t = peek();
    if (t.type == TokenType::RETURN)   return parse_return();
    if (t.type == TokenType::BREAK)    return parse_break();
    if (t.type == TokenType::CONTINUE) return parse_continue();
    if (t.type == TokenType::IF)       return parse_if();
    if (t.type == TokenType::WHILE)    return parse_while();
    if (t.type == TokenType::FOR)      return parse_for();
    if (t.type == TokenType::L_CURLB)  return parse_block();
    if (t.type == TokenType::LET)      return parse_variable_declaration();
    if (t.type == TokenType::FUNCTION) return parse_function();
    if (t.type == TokenType::CLASS)    return parse_blueprint();
    if (t.type == TokenType::PRINT)    return parse_print();
    if (t.type == TokenType::SCAN)     return parse_scan();

    auto expr = parse_expression(0);
    consume_if(TokenType::SEMICOLON, "Expected ';'");
    return make_unique<ExprStmt>(std::move(expr));
}
```

The **central routing function.** Looks at the current token and decides which specific parse function to call. If none match, it tries to parse an expression statement (like `x = 5;` or `foo();`).

Specific Statement Parsers

`parse_variable_declaration()` — handles `let x: int = 5;`

cpp

```
consume();           // eat 'let'
Token nameToken = consume();  // eat 'x'
consume_if(OP_COLON, "Expected ':'");
Type type = parse_type();    // eat 'int'
consume_if(ASSIGN, "Expected '='");
auto initializer = parse_expression(0);  // eat '5'
consume_if(SEMICOLON, "Expected ';'");
return make_unique<VarDeclStmt>(nameToken.value, type, move(initializer));
```

`parse_function()` — handles `function add(let a: int, let b: int): int { ... }`

cpp

```
consume();  // eat 'function'
Token name = consume();  // eat function name
consume_if(L_PAREN, "Expected '('");
// Parse parameters in a loop
vector<Parameter> args;
while (peek() != R_PAREN) {
    args.push_back(parse_parameter());
    if (peek() == COMMA) consume();
}
consume_if(R_PAREN, "Expected ')'");
consume_if(OP_COLON, "Expected ':'");
Type func_type = parse_type();  // Return type
auto body = parse_block();
return make_unique<FunctionStmt>(...);
```

`parse_if()` — handles `if (condition) { ... } else { ... }`

cpp

```
consume();  // eat 'if'
consume_if(L_PAREN, ...);
auto condition = parse_expression(0);
consume_if(R_PAREN, ...);
auto thenBranch = parse_statement();
unique_ptr<Stmt> elseBranch = nullptr;
if (peek().type == ELSE) { consume(); elseBranch = parse_statement(); }
return make_unique<IfStmt>(move(condition), move(thenBranch), move(elseBranch));
```

`parse_expression()` — Pratt Parser

This is the most sophisticated function in the Parser, implementing a **Pratt (top-down operator precedence) parser**.

cpp

```
unique_ptr<Expr> Parser::parse_expression(int min_bp) {
    Token t = consume();
    unique_ptr<Expr> lhs;
```

`min_bp` = minimum binding power. Operators with binding power below this will not be consumed (they belong to a higher-level expression).

Prefix parsing (building the left-hand side):

```cpp
    if (t.type == TokenType::MULTI) {
        auto rhs = parse_expression(29);
        lhs = make_unique<DereferenceExpr>(move(rhs));  // *ptr
    }
    else if (t.type == MINUS || t.type == NOT || ...) {
        auto [l,r] = get_binding_power(t.type);
        auto rhs = parse_expression(r);
        lhs = make_unique<UnaryExpr>(t, move(rhs), false);  // -x, !x
    }
    else if (is a literal) lhs = make_unique<LiteralExpr>(t);
    else if (t.type == L_PAREN) { lhs = parse_expression(0); consume(); }  // (expr)
    else if (t.type == VAL) lhs = make_unique<VariableExpr>(t);  // variable
    else if (t.type == NEW) { ... lhs = make_unique<InstantiateExpr>(...); }  // new
```

Infix parsing loop (extending the expression with operators):

```cpp
    while (true) {
        Token op = peek();
        // Handle postfix: obj.field, arr[i], func()
        if (peek().type == DOT || peek().type == ARROW) { ... lhs = make_unique<Memb
        if (peek().type == L_SQUB) { ... lhs = make_unique<ArrayAccessExpr>(...); }
        if (peek().type == L_PAREN) { ... lhs = make_unique<CallExpr>(...); }

        // Stop if operator has too low precedence
        auto [l_bp, r_bp] = get_binding_power(op.type);
        if (l_bp == 0 || l_bp < min_bp) break;
        consume();

        // Assignment or binary expression
        if (is assignment op) lhs = make_unique<AssignExpr>(move(lhs), op, parse_exp
        else lhs = make_unique<BinaryExpr>(move(lhs), op, parse_expression(r_bp));
    }
    return lhs;
```

The loop **continuously extends** the expression. For `2 + 3 * 4`, it builds `2`, then sees `+` (bp=23), recurses with `min_bp=24` which grabs `3 * 4` as a unit, then returns `BinaryExpr(2, +, BinaryExpr(3, *, 4))`.

---

5. ASTPrinter.h — The AST Printer

This is a **debugging tool** that walks the AST and prints it in a readable tree format. It uses the **Visitor pattern**.

cpp

```cpp
class ASTPrinter : public Visitor {
private:
    int indent_level = 0;

    void printIndent() {
        for (int i = 0; i < indent_level; i++) cout << "  | ";
    }
```

`indent_level` tracks how deep in the tree we are. `printIndent()` prints `  |` for each level, creating a visual tree structure.

cpp

```cpp
    void visit(ProgramNode* node) override {
        cout << "Program\n";
        indent_level++;
        for (auto& stmt : node->statements) stmt->accept(this);
        indent_level--;
    }
```

For each node type, the pattern is: **print the node name → increase indent → visit children → decrease indent**. This creates the tree structure.

cpp

```cpp
    void visit(BinaryExpr* expr) override {
        printIndent(); cout << "BinaryExpr (op: " << expr->op.value << ")\n";
        indent_level++;
        expr->left->accept(this);
        expr->right->accept(this);
        indent_level--;
    }
```

For `2 + 3`, this prints:

```
BinaryExpr (op: +)
  | Literal (2)
  | Literal (3)
```

Unimplemented nodes (stubs):

cpp

```cpp
    void visit(WhileStmt* stmt) override {}
    void visit(ForStmt* stmt) override {}
    // ...
```

These are empty overrides to prevent compile errors for Visitor methods that the printer doesn't need to display. They satisfy the abstract Visitor interface.

---

6. SemanticAnalyzer.h & SemanticAnalyzer.cpp

The Semantic Analyzer is the **type checker and scope validator**. After the parser builds the AST, this stage walks it and verifies that the code makes logical sense — right types, no undeclared variables, no break outside loops, etc.

SemanticAnalyzer.h — The Header

Supporting Data Structures:

cpp

```cpp
struct FunctionType {
    ::Type returnType;
    vector<::Type> paramTypes;
};
```

Stores the complete signature of a function — what it returns and what types its parameters expect.

cpp

```cpp
struct Symbol {
    string_view name;
    ::Type type;
    bool isFunction;
    ::FunctionType funcType;
    unordered_map<string, ::Type> blueprintFields;
};
```

An entry in the symbol table. Every declared variable, function, or blueprint is stored as a `Symbol`. The `blueprintFields` map stores the fields of a Blueprint definition.

The Class:

```cpp
class SemanticAnalyzer : public Visitor {
private:
    vector<unordered_map<string_view, Symbol>> scopes;  // Symbol table stack
    ::Type current_type;                                  // Carries expression type
    int loop_depth = 0;                                   // Tracks if we're inside
```

- `scopes` is a **stack of symbol tables**. Each block `{ }` pushes a new map; exiting it pops the map. This implements **lexical scoping**.
- `current_type` is the visitor return value workaround — since `visit()` returns `void`, the type of an evaluated expression is stored here and read by the parent node.
- `loop_depth` tracks nested loop depth so `break`/`continue` can be validated.

SemanticAnalyzer.cpp — The Implementation

Scope Management

```cpp
void SemanticAnalyzer::enter_scope() { scopes.push_back({}); }

void SemanticAnalyzer::exit_scope() {
    if (scopes.size() <= 1) throw_error("Cannot exit the global scope!");
    scopes.pop_back();
}
```

Push/pop maps on the `scopes` vector. We never pop the global scope (size guard).

```cpp
Symbol* SemanticAnalyzer::lookup(string_view name) {
    for (auto it = scopes.rbegin(); it != scopes.rend(); ++it) {
        auto found = it->find(name);
        if (found != it->end()) return &(found->second);
    }
    throw runtime_error("Error: variable '" + string(name) + "' doesn't exist");
}
```

Searches **from the innermost scope outward** (using reverse iterator `rbegin`). This ensures inner variables shadow outer ones — the closest declaration wins.

```cpp
void SemanticAnalyzer::declare(string_view name, Type type, bool isFunc, FunctionTyp
    auto& currentScope = scopes.back();
    auto found = currentScope.find(name);
    if (found != currentScope.end())
        throw runtime_error("Error: variable '" + string(name) + "' already exists")
    currentScope[name] = Symbol{name, type, isFunc, fType};
}
```

Declares a symbol in the **current (innermost) scope only**. Throws if a duplicate is found in the same scope.

Expression Visitors

```cpp
void SemanticAnalyzer::visit(LiteralExpr* expr) {
    switch(expr->value.type) {
        case TokenType::INT_LIT:    current_type = Type(TokenType::INT); break;
        case TokenType::FLOAT_LIT:  current_type = Type(TokenType::FLOAT); break;
        case TokenType::STRING_LIT: current_type = Type(TokenType::STRING); break;
        // ...
    }
}
```

Sets `current_type` based on the literal token type. `42` → INT, `"hello"` → STRING, etc.

```cpp
```

```cpp
void SemanticAnalyzer::visit(VariableExpr* expr) {
    Symbol* symbol = lookup(expr->name.value);
    current_type = symbol->type;
}
```

Looks up the variable and propagates its type upward. If the variable doesn't exist, `lookup()` throws.

cpp

```cpp
void SemanticAnalyzer::visit(BinaryExpr* expr) {
    expr->left->accept(this);
    Type leftType = current_type;
    expr->right->accept(this);
    Type rightType = current_type;

    if (leftType != rightType)
        throw_error("Type mismatch: Cannot perform operation between different types

    TokenType op = expr->op.type;

    if (op == PLUS || op == MINUS || op == MULTI || op == DIV || op == MOD) {
        if (leftType.base != INT && leftType.base != FLOAT && leftType.base != DOUBL
            throw_error("Math operations are only allowed on numbers.");
        current_type = leftType;  // Result is same type as operands
    }
    else if (comparison operators) {
        current_type = Type(TokenType::BOOLEAN);  // Comparisons always produce bool
    }
    else if (op == AND || op == OR) {
        if (leftType.base != BOOLEAN) throw_error("&& and || require booleans.");
        current_type = Type(TokenType::BOOLEAN);
    }
}
```

The type-checking logic for binary expressions. Validates both sides are compatible, then sets the result type.

Statement Visitors

cpp

```
void SemanticAnalyzer::visit(VarDeclStmt* stmt) {
    stmt->initializer->accept(this);  // Evaluate initializer type
    Type valueType = current_type;
    if (stmt->type != valueType)
        throw_error("Type mismatch in variable declaration: '" + string(stmt->name)
    declare(stmt->name, stmt->type);  // Register in current scope
}
```

Verifies that the declared type matches the initializer's type (`let x: int = "hello"` would throw).

cpp

```
void SemanticAnalyzer::visit(IfStmt* stmt) {
    stmt->condition->accept(this);
    if (current_type.base != TokenType::BOOLEAN)
        throw_error("If condition must evaluate to a boolean type.");
    stmt->thenBranch->accept(this);
    if (stmt->elseBranch) stmt->elseBranch->accept(this);
}
```

Enforces that `if` conditions are boolean expressions.

cpp

```
void SemanticAnalyzer::visit(WhileStmt* stmt) {
    stmt->condition->accept(this);
    if (current_type.base != TokenType::BOOLEAN)
        throw_error("While condition must be boolean.");
    loop_depth++;          // We are now inside a loop
    stmt->body->accept(this);
    loop_depth--;          // We exited the loop
}

void SemanticAnalyzer::visit(BreakStmt* stmt) {
    if (loop_depth == 0) throw_error("'break' can only be used inside a loop!");
}
```

`loop_depth` is incremented when entering a loop and decremented when leaving. `break` and `continue` check that `loop_depth > 0`.

cpp

```
void SemanticAnalyzer::visit(FunctionStmt* stmt) {
    FunctionType fType;
    fType.returnType = stmt->type;
    for (auto& p : stmt->params) fType.paramTypes.push_back(p.type);

    declare(stmt->name, stmt->type, true, fType);  // Register function in outer sco
    enter_scope();                                  // New scope for parameters
    for (auto& p : stmt->params) declare(p.name, p.type);
    stmt->body->accept(this);
    exit_scope();
}
```

Registers the function in the current scope (so it can be called later), then creates a new scope for the parameters and body.

cpp

```
void SemanticAnalyzer::visit(CallExpr* expr) {
    VariableExpr* varCallee = dynamic_cast<VariableExpr*>(expr->callee.get());
    Symbol* sym = lookup(varCallee->name.value);
    if (!sym->isFunction) throw_error("Not a function!");

    FunctionType fType = sym->funcType;
    if (expr->args.size() != fType.paramTypes.size())
        throw_error("Incorrect number of arguments to: " + string(sym->name));

    for (size_t i = 0; i < expr->args.size(); i++) {
        expr->args[i]->accept(this);
        if (current_type != fType.paramTypes[i])
            throw_error("Type mismatch in parameters of: " + string(sym->name));
    }
    current_type = fType.returnType;  // Expression type = function's return type
}
```

Validates function calls: the callee must be a declared function, argument count must match, and each argument's type must match the parameter type.

---

7. IRGenerator.h & IRGenerator.cpp

The IR Generator is the **final and most complex stage**. It walks the AST and emits **LLVM IR** — a portable assembly-like language that LLVM can then compile to native machine code for any

platform.

IRGenerator.h — The Header

```cpp
class IRGenerator : public Visitor {
public:
    unique_ptr<LLVMContext> context;    // Global LLVM context (owns all types/const
    unique_ptr<Module> module;          // The LLVM module = one compiled file
    unique_ptr<IRBuilder<>> builder;    // The instruction emitter
```

The three core LLVM objects:

- `LLVMContext` — holds all global LLVM state. Think of it as the "universe" for all LLVM objects.
- `Module` — the container for all generated code (functions, globals). It's the output.
- `IRBuilder` — a helper that makes it easy to emit LLVM instructions at the current insertion point.

```cpp
    map<string, AllocaInst*> NamedValues;         // Variable name → memory location
    map<string, llvm::StructType*> StructTypes;   // Blueprint name → LLVM struct ty
    map<string, map<string, int>> StructFields;   // Blueprint name → (field name →

    vector<BasicBlock*> LoopIncBlocks;   // Stack of 'continue' targets
    vector<BasicBlock*> LoopExitBlocks;  // Stack of 'break' targets

    Value* currentVal = nullptr;  // Carries LLVM Values up the AST tree
```

- `NamedValues` is the **IR-level symbol table** mapping variable names to their `AllocaInst` (stack memory allocations).
- `StructTypes` and `StructFields` store Blueprint (struct) definitions.
- `LoopIncBlocks`/`LoopExitBlocks` are stacks that track where `continue` and `break` should jump — necessary for nested loops.
- `currentVal` is the equivalent of `current_type` in the SemanticAnalyzer — it carries generated LLVM `Value*` objects up the visitor tree.

IRGenerator.cpp — The Implementation

Helper: `processEscapes()`

```cpp
string processEscapes(string input) {
    string res;
    for (size_t i = 0; i < input.length(); ++i) {
        if (input[i] == '\\' && i+1 < input.length()) {
            switch(input[++i]) {
                case 'n': res += '\n'; break;
                case 't': res += '\t'; break;
                case '"': res += '"'; break;
                default:  res += input[i]; break;
            }
        } else res += input[i];
    }
    return res;
}
```

Converts escape sequences in string literals. When the source code contains `\n`, the lexer stores it as two characters `\` and `n`. This function replaces them with the actual newline character.

`getLLVMType()` — Type Converter

```cpp
llvm::Type* IRGenerator::getLLVMType(TokenType kind) {
    if (kind == TokenType::INT)     return llvm::Type::getInt32Ty(*context);    // 32
    if (kind == TokenType::FLOAT)   return llvm::Type::getInt32Ty(*context);    // No
    if (kind == TokenType::DOUBLE)  return llvm::Type::getInt32Ty(*context);    // No
    if (kind == TokenType::BOOLEAN) return llvm::Type::getInt1Ty(*context);     // 1-
    if (kind == TokenType::CHAR)    return llvm::Type::getInt8Ty(*context);     // 8-
    if (kind == TokenType::STRING)  return llvm::PointerType::get(*context,0); // Op
    if (kind == TokenType::CLASS)   return llvm::PointerType::get(*context,0); // Op
    if (kind == TokenType::VOID)    return llvm::Type::getVoidTy(*context);
    return nullptr;
}
```

Maps your language's type system to LLVM's type system.

Constructor

```cpp
IRGenerator::IRGenerator() {
    context = make_unique<LLVMContext>();
    module  = make_unique<Module>("myCompiler", *context);
    builder = make_unique<IRBuilder<>>(*context);

    // Pre-declare printf and scanf (C standard library functions)
    vector<llvm::Type*> ioArgs;
    ioArgs.push_back(llvm::PointerType::get(*context, 0));  // char* format string
    llvm::FunctionType* ioType = llvm::FunctionType::get(llvm::Type::getInt32Ty(*con
    // 'true' means variadic (printf takes variable arguments)
    llvm::Function::Create(ioType, llvm::Function::ExternalLinkage, "printf", module
    llvm::Function::Create(ioType, llvm::Function::ExternalLinkage, "scanf",  module
}
```

Initializes all three LLVM objects and **pre-declares** `printf` and `scanf` as external functions so they can be called by the generated code. `ExternalLinkage` means "this function lives in another library (libc)".

`generate()` — Entry Point

```cpp
Value* IRGenerator::generate(Node* node) {
    node->accept(this);   // Triggers visit() via Visitor pattern
    return currentVal;    // Returns whatever was generated
}
```

A thin wrapper that triggers the Visitor dispatch and returns the resulting LLVM value.

`visit(LiteralExpr*)` — Literal Code Generation

```cpp
```

```cpp
void IRGenerator::visit(LiteralExpr* expr) {
    string lexeme = string(expr->value.value);
    switch(type) {
        case INT_LIT:
            currentVal = ConstantInt::get(*context, APInt(32, stoi(lexeme), true));
            break;
        case FLOAT_LIT:
            currentVal = ConstantFP::get(*context, APFloat(stod(lexeme)));
            break;
        case BOOLEAN_LIT:
            currentVal = ConstantInt::get(*context, APInt(1, lexeme=="true", false))
            break;
        case CHAR_LIT:
            currentVal = ConstantInt::get(*context, APInt(8, lexeme[1], true));
            break;
        case STRING_LIT:
            string content = lexeme.substr(1, lexeme.length()-2);  // Strip quotes
            currentVal = builder->CreateGlobalString(processEscapes(content), "strtm
            break;
    }
}
```

Creates LLVM constant values. For example, `42` becomes a 32-bit LLVM integer constant. String literals become global constant arrays (stored in the data section of the binary).

`visit(VariableExpr*)` — Variable Access

```cpp
void IRGenerator::visit(VariableExpr* expr) {
    AllocaInst* alloca = NamedValues[varName];
    currentVal = builder->CreateLoad(alloca->getAllocatedType(), alloca, varName.c_s
}
```

Loads the **value** from the variable's memory location. In LLVM IR, variables live on the stack (`AllocaInst`) and must be explicitly loaded (`CreateLoad`) to get their value.

`visit(BinaryExpr*)` — Binary Operations

```cpp
```

```cpp
void IRGenerator::visit(BinaryExpr* expr) {
    expr->left->accept(this);  Value* l = currentVal;
    expr->right->accept(this); Value* r = currentVal;

    switch (expr->op.type) {
        case PLUS:  currentVal = builder->CreateAdd(l, r, "addtmp"); break;
        case MINUS: currentVal = builder->CreateSub(l, r, "subtmp"); break;
        case MULTI: currentVal = builder->CreateMul(l, r, "addtmp"); break;
        case DIV:   currentVal = builder->CreateSDiv(l, r, "subtmp"); break;  // Sig
        case LESSER:        currentVal = builder->CreateICmpSLT(l, r, "cmptmp"); brea
        case GREATER:       currentVal = builder->CreateICmpSGT(l, r, "cmptmp"); brea
        case EQUALS:        currentVal = builder->CreateICmpEQ(l, r, "cmptmp"); break
        case NOT_EQU:       currentVal = builder->CreateICmpNE(l, r, "cmptmp"); break
        case LESSER_EQU:    currentVal = builder->CreateICmpSLE(l, r, "cmptmp"); brea
        case GREATER_EQU:   currentVal = builder->CreateICmpSGE(l, r, "cmptmp"); brea
        case AND: currentVal = builder->CreateAnd(l, r, "cmptmp"); break;
        case OR:  currentVal = builder->CreateOr(l, r, "cmptmp"); break;
    }
}
```

Generates arithmetic and comparison instructions. `ICmp` = integer compare, `S` = signed. Comparison instructions return LLVM `i1` (1-bit boolean) values.

`visit(AssignExpr*)` — Assignment

cpp

```cpp
void IRGenerator::visit(AssignExpr* expr) {
    expr->value->accept(this);
    llvm::Value* val = currentVal;

    llvm::Value* targetPtr = nullptr;

    if (auto* varExpr = dynamic_cast<VariableExpr*>(expr->target.get())) {
        targetPtr = NamedValues[string(varExpr->name.value)];  // Simple variable
    }
    else if (auto* memExpr = dynamic_cast<MemberExpr*>(expr->target.get())) {
        // Calculate GEP (Get Element Pointer) for struct field
        memExpr->object->accept(this);
        llvm::Value* basePtr = currentVal;
        int fieldIndex = StructFields[structName][propertyName];
        vector<Value*> indices = { ConstantInt::get(0), ConstantInt::get(fieldIndex)
        targetPtr = builder->CreateInBoundsGEP(StructTypes[structName], basePtr, ind
    }

    // Handle compound assignments (+=, -=, etc.)
    switch(expr->op.type) {
        case PLUS_EQU: {
            Value* curVal = builder->CreateLoad(val->getType(), targetPtr, "loadtmp"
            val = builder->CreateAdd(curVal, val, "addtmp");
            break;
        }
        // ... similar for -=, *=, /=
        case ASSIGN: break;  // Simple assignment, nothing to do
    }

    builder->CreateStore(val, targetPtr);  // Write the value to memory
    currentVal = val;
}
```

Assignment must find the **memory address** (`targetPtr`) to write to. For simple variables it's their `AllocaInst`. For struct fields it uses GEP (GetElementPtr) — LLVM's instruction for computing field addresses within a struct.

`visit(VarDeclStmt*)` — Variable Declaration

```cpp
```

```cpp
void IRGenerator::visit(VarDeclStmt* stmt) {
    llvm::Type* llvmType = getLLVMType(stmt->type.base);
    AllocaInst* alloca = builder->CreateAlloca(llvmType, nullptr, varName);  // Stac
    NamedValues[varName] = alloca;  // Register in symbol table
    if (stmt->initializer) {
        stmt->initializer->accept(this);
        builder->CreateStore(currentVal, alloca);  // Store initial value
    }
}
```

`CreateAlloca` reserves space on the stack. `CreateStore` writes the initial value. This is the LLVM way to create local variables.

`visit(IfStmt*)` — If/Else Code Generation

cpp

```cpp
void IRGenerator::visit(IfStmt* stmt) {
    stmt->condition->accept(this);
    Value* cond = builder->CreateICmpNE(currentVal, ConstantInt::get(cond->getType()
    // Convert to i1 boolean by comparing with 0

    Function* theFunction = builder->GetInsertBlock()->getParent();

    BasicBlock* thenBB  = BasicBlock::Create(*context, "then", theFunction);
    BasicBlock* elseBB  = BasicBlock::Create(*context, "else");    // Not attached y
    BasicBlock* mergeBB = BasicBlock::Create(*context, "ifcont");  // Not attached y

    builder->CreateCondBr(cond, thenBB, elseBB);  // Conditional branch

    // Generate THEN branch
    builder->SetInsertPoint(thenBB);
    stmt->thenBranch->accept(this);
    builder->CreateBr(mergeBB);  // Jump to merge after then

    // Attach else block and generate ELSE branch
    theFunction->insert(theFunction->end(), elseBB);
    builder->SetInsertPoint(elseBB);
    if (stmt->elseBranch) stmt->elseBranch->accept(this);
    builder->CreateBr(mergeBB);

    // Continue after if/else
    theFunction->insert(theFunction->end(), mergeBB);
    builder->SetInsertPoint(mergeBB);
}
```

In LLVM, if/else is implemented with **basic blocks** and conditional branches. Three blocks are created: `then`, `else`, and `merge` (where both branches meet). `CreateCondBr` is the LLVM conditional branch instruction.

`visit(WhileStmt*)` — While Loop

```cpp
```

```cpp
void IRGenerator::visit(WhileStmt* stmt) {
    BasicBlock* condBB = BasicBlock::Create(*context, "while.cond", theFunction);
    BasicBlock* bodyBB = BasicBlock::Create(*context, "while.body", theFunction);
    BasicBlock* endBB  = BasicBlock::Create(*context, "while.end",  theFunction);

    builder->CreateBr(condBB);           // Jump into condition block
    builder->SetInsertPoint(condBB);
    stmt->condition->accept(this);       // Evaluate condition every loop iteration
    cond = builder->CreateICmpNE(cond, ConstantInt::get(0), "whilecond");
    builder->CreateCondBr(cond, bodyBB, endBB);  // Branch based on condition

    LoopIncBlocks.push_back(condBB);   // 'continue' goes back to condition
    LoopExitBlocks.push_back(endBB);   // 'break' goes to end

    builder->SetInsertPoint(bodyBB);
    stmt->body->accept(this);
    builder->CreateBr(condBB);         // Jump back to condition after body

    LoopIncBlocks.pop_back();
    LoopExitBlocks.pop_back();
    builder->SetInsertPoint(endBB);    // Code after while continues here
}
```

Three blocks: **condition** (evaluated each iteration), **body** (the loop contents), **end** (after the loop). `continue` jumps back to `condBB`; `break` jumps to `endBB`.

`visit(ForStmt*)` — For Loop

Similar to while but adds a fourth block `for.inc` for the increment expression. Order: `cond → body → inc → cond → ...`

`visit(FunctionStmt*)` — Function Definition

cpp

```cpp
void IRGenerator::visit(FunctionStmt* stmt) {
    llvm::Type* returnType = getLLVMType(stmt->type.base);
    vector<llvm::Type*> argTypes;
    for (auto& param : stmt->params) argTypes.push_back(getLLVMType(param.type.base)

    FunctionType* funcType = FunctionType::get(returnType, argTypes, false);
    Function* theFunction = Function::Create(funcType, Function::ExternalLinkage, st

    BasicBlock* entrybb = BasicBlock::Create(*context, "entry", theFunction);
    builder->SetInsertPoint(entrybb);  // Start writing instructions here

    NamedValues.clear();  // New scope — clear old variables
    unsigned idx = 0;
    for (auto& arg : theFunction->args()) {
        string argName = string(stmt->params[idx].name);
        arg.setName(argName);
        AllocaInst* alloca = builder->CreateAlloca(argTypes[idx], nullptr, argName);
        builder->CreateStore(&arg, alloca);  // Copy argument into local variable
        NamedValues[argName] = alloca;
        idx++;
    }

    stmt->body->accept(this);
    if (returnType->isVoidTy()) builder->CreateRetVoid();
    verifyFunction(*theFunction);  // LLVM validates the function is well-formed
}
```

Creates an LLVM function, creates an `entry` basic block, copies parameters into local stack variables (LLVM's standard way), then generates the body. `verifyFunction` catches any IR errors.

`visit(BluePrintExpr*)` — Blueprint (Struct) Definition

```cpp
```

```
void IRGenerator::visit(BluePrintExpr* stmt) {
    vector<llvm::Type*> bodyTypes;
    map<string, int> fields;
    int currentIndex = 0;

    for (auto& fieldNode : stmt->feild) {
        auto* varDecl = dynamic_cast<VarDeclStmt*>(fieldNode.get());
        // Map each field to its LLVM type
        llvm::Type* fieldType = ...;
        bodyTypes.push_back(fieldType);
        fields[string(varDecl->name)] = currentIndex++;  // Remember field index
    }

    StructType* structType = StructType::create(*context, bodyTypes, structName);
    StructTypes[structName] = structType;   // Remember for GEP calculations
    StructFields[structName] = fields;
}
```

Creates an LLVM `StructType` from the Blueprint's fields. Field indices are stored so `MemberExpr` can compute the correct GEP offset later.

`visit(PrintStmt*)` — Print Statement

cpp

```
void IRGenerator::visit(PrintStmt* stmt) {
    stmt->value->accept(this);
    llvm::Value* val = currentVal;

    llvm::Value* formatStr = nullptr;
    if (val->getType()->isIntegerTy(32))
        formatStr = builder->CreateGlobalString("%d\n", "printInt");   // Integer fo
    else if (val->getType()->isPointerTy())
        formatStr = builder->CreateGlobalString("%s\n", "printStr");   // String for

    llvm::Function* printfFunc = module->getFunction("printf");
    builder->CreateCall(printfFunc, {formatStr, val}, "printfCall");
}
```

Your `print()` statement compiles to a `printf` call. It auto-selects the format string (`%d` for integers, `%s` for strings) based on the LLVM type of the value.

`visit(ScanStmt*)` — Scan Statement

```cpp
void IRGenerator::visit(ScanStmt* stmt) {
    llvm::AllocaInst* alloca = NamedValues[varName];
    llvm::Value* formatStr = builder->CreateGlobalString("%d\n", "scanFmt");
    llvm::Function* scanfFunc = module->getFunction("scanf");
    builder->CreateCall(scanfFunc, {formatStr, alloca}, "scanfCall");
}
```

Your `scan()` statement compiles to a `scanf` call. Crucially, it passes the **memory address** (`alloca`) of the variable (not its value), because `scanf` writes into the address it receives.

---

8. How All Parts Work Together

Here's the complete flow for compiling `let x: int = 5 + 3;`:

Stage	Input	Action	Output
Lexer	`let x: int = 5 + 3;`	Tokenizes char by char	`LET, VAL("x"), COLON, INT, ASSIGN, INT_LIT("5"), PLUS, INT_LIT("3"), SEMICOLON`
Parser	Token list	Builds tree using Pratt parsing	`VarDeclStmt{ name="x", type=INT, init=BinaryExpr{5, +, 3} }`
AST Printer	AST	Walks tree, prints structure	`VarDecl (name: x)` → `BinaryExpr (+)` → `Literal(5)`, `Literal(3)`
Semantic Analyzer	AST	Type-checks: `INT + INT = INT`, matches declared type `INT`	✓ Passes
IR Generator	AST	Generates LLVM instructions	`%x = alloca i32`, `store i32 8, %x`

The Visitor Pattern

Every stage (ASTPrinter, SemanticAnalyzer, IRGenerator) uses the **Visitor design pattern**. Each AST node has an `accept(Visitor* v)` method that calls `v->visit(this)`. This means:

- The AST nodes don't need to know about printing, type-checking, or IR generation

- You can add new passes (optimization, transpilation) without changing any AST node
- Each stage is completely self-contained

Memory Management

All AST nodes are stored as `unique_ptr<>`. When the root `ProgramNode` is destroyed, it automatically destroys all its children — no memory leaks. LLVM objects are managed by `context` and `module`, which own everything allocated through them.

This compiler implements all core stages of a production compiler: lexing → parsing → semantic analysis → code generation. The output LLVM IR can be compiled to native machine code on any platform LLVM supports.